LibSpace

Digital Library Management API

Project Documentation

Version 1.0.0

Node.js · Express · MongoDB · JWT Authentication

TEAM MEMBERS

1. **GOPINATH L**
2. **AMARNATH DS**
3. **EBENEZER P**
4. **AJAYCHANDRAN C**

1. Project Overview

LibSpace (My-LibSpace) is a RESTful backend API designed as an educational project to teach students the fundamentals of authentication and CRUD operations. Built with Node.js, Express, and MongoDB, it provides a secure digital library management system where authenticated users can manage their personal book collections.

1.1 Purpose & Goals

- Demonstrate JWT-based authentication (register, login, logout) with token blacklisting
- Implement full CRUD operations on a Book resource
- Enforce per-user data isolation so each user sees only their own books
- Serve as a learning reference for Node.js REST API best practices

1.2 Key Features

Feature	Description
User Registration	New users sign up with name, email, and password (bcrypt-hashed)
User Login	Returns a signed JWT valid for 1 hour upon correct credentials
Logout / Token Blacklisting	Invalidates JWT by storing it in a MongoDB blacklist collection
Account Deletion	Authenticated users can permanently delete their account
Add Book	Authenticated users add books with title, author, and URL
Get All Books	Returns all books belonging to the logged-in user
Get Book by ID	Fetches a single book by its ID (user-scoped)
Update Book	Updates any book field via partial update (PATCH-style PUT)
Delete Book	Removes a book by ID (user-scoped)

2. Technology Stack

Technology	Version	Purpose
Node.js	v14+	JavaScript runtime environment
Express	^5.2.1	Web framework for routing and middleware
MongoDB	Cloud / Local	NoSQL database for persistent storage

Technology	Version	Purpose
Mongoose	^9.2.1	ODM library for MongoDB schema management
jsonwebtoken	^9.0.3	JWT creation and verification
bcrypt	^6.0.0	Password hashing (salt rounds: 10)
dotenv	^17.3.1	Environment variable management
cors	^2.8.6	Cross-Origin Resource Sharing support
nodemon	^3.1.14	Dev server auto-restart on file changes

3. Project Structure

The project follows a standard MVC (Model-View-Controller) architecture with the following directory layout:

```
my-libspace/
├── Config/
│   └── db.js                  # MongoDB connection setup
├── controller/
│   ├── authController.js       # User auth logic
(register/login/logout/delete)
│   └── bookController.js       # Book CRUD logic
├── middlewares/
│   └── verify_token.js         # JWT verification + blacklist check
├── models/
│   ├── user.model.js           # Mongoose User schema
│   ├── book.model.js           # Mongoose Book schema
│   └── blackListerToken.js     # Mongoose Blacklist schema
├── routes/
│   ├── user.route.js           # Auth route definitions
│   └── book.route.js           # Book route definitions
├── index.js                    # App entry point
├── package.json
└── .env                        # Environment variables (not committed)
```

4. Data Models

4.1 User Model

Stored in the users MongoDB collection. Tracks creation/update timestamps automatically.

Field	Type	Required	Description
name	String	Yes	Display name of the user
email	String	Yes	Unique email address for login
password	String	Yes	bcrypt-hashed password (10 salt rounds)
createdAt	Date	Auto	Timestamp added by Mongoose timestamps option
updatedAt	Date	Auto	Timestamp added by Mongoose timestamps option

4.2 Book Model

Stores book entries linked to the owning user via a reference ObjectId.

Field	Type	Required	Description
url	String	Yes	URL to the book resource or cover image
title	String	Yes	Title of the book (unique enforcement in controller)
author	String	Yes	Author name
user	ObjectId (ref: User)	Yes	Reference to the owning user — enforces isolation

4.3 BlackList Token Model

Stores invalidated JWTs. When a user logs out, their token is saved here. The verifyToken middleware checks this collection before granting access to protected routes.

Field	Type	Description
token	String	The full JWT string to be blocked

5. API Endpoints

5.1 Authentication Routes

Base path: /api/authuser (also /api for legacy compatibility)

Method	Endpoint	Auth	Description
POST	/api/authuser/register	None	Register a new user account
POST	/api/authuser/login	None	Login and receive a JWT token

Method	Endpoint	Auth	Description
POST	/api/authuser/logout	Bearer Token	Invalidate (blacklist) the current JWT
DELETE	/api/authuser/:userId	Bearer Token	Delete the user account permanently

5.2 Book Routes

Base path: /api/book — All routes require a valid Bearer token.

Method	Endpoint	Description
POST	/api/book/addbook	Add a new book (title must be unique)
GET	/api/book/getBooks	Retrieve all books for the logged-in user
GET	/api/book/:bookId	Retrieve a single book by ID (user-scoped)
PUT	/api/book/:bookId	Update any fields of an existing book
DELETE	/api/book/deleteBook/:bookId	Delete a book by ID (user-scoped)

5.3 Request & Response Examples

Register User — POST /api/authuser/register

Request body:

```
{ "name": "Alice", "email": "alice@example.com", "password": "secret123" }
```

Success response:

```
{ "success": true, "message": "User registered successfully!" }
```

Login User — POST /api/authuser/login

Request body:

```
{ "email": "alice@example.com", "password": "secret123" }
```

Success response (200):

```
{ "msg": "Logged In Successfully", "token": "<JWT>", "user": [...] }
```

Add Book — POST /api/book/addbook

Headers: Authorization: Bearer <token>

Request body:

```
{ "title": "Clean Code", "author": "Robert C. Martin", "url": "https://..." }
```

Success response:

```
{ "msg": "Book added successfully" }
```

6. Authentication & Security

6.1 Authentication Flow

- 1. User registers — password is hashed with bcrypt (10 salt rounds) before storage.
- 2. User logs in — bcrypt.compare() validates the password; a JWT is signed with the user's id and email, valid for 1 hour.
- 3. Protected requests — client sends the JWT as a Bearer token in the Authorization header.
- 4. verifyToken middleware — checks the blacklist first, then calls jwt.verify() with the SECRET_KEY.
- 5. User logs out — the token is saved to the BlackList collection; subsequent use returns 401.

6.2 JWT Payload

```
{ "id": "<MongoDB ObjectId>", "email": "alice@example.com", "iat": ..., "exp":
... }
```

6.3 Security Considerations

- Passwords are never stored in plaintext — bcrypt hashing is enforced at registration.
- JWT expiry is set to 1 hour to limit the window for stolen tokens.
- Token blacklisting on logout prevents re-use of valid tokens after sign-out.
- All book operations are user-scoped — queries always include the requesting user's ID to prevent cross-user data access.
- The SECRET_KEY and MONGO_URI are stored in .env and never committed to source control.

7. Setup & Installation

7.1 Prerequisites

- Node.js v14 or higher
- npm or yarn package manager
- MongoDB instance (local or MongoDB Atlas cloud)
- A code editor (VS Code recommended)

7.2 Installation Steps

Step 1 — Clone the repository:

```
git clone https://github.com/divya-mephisto/Libspace-NM
```

```
cd My-LibSpace
```

Step 2 — Install dependencies:

```
npm install
```

Step 3 — Create the .env file in the project root:

```
MONGO_URI=mongodb+srv://<user>:<password>@cluster.mongodb.net/libspace
PORT=5000
JWT_SECRET=your_super_secret_key_here
```

Step 4 — Start the development server:

```
npm run start
```

The server will start on the configured PORT (default: 5000). You should see:

```
Our server is working at PORT :5000
Connected to mongodb
```

7.3 Environment Variables Reference

Variable	Required	Description
MONGO_URI	Yes	MongoDB connection string (Atlas or local)
PORT	No	Server port (defaults to 5000 if not set)
JWT_SECRET	Yes	Secret key used to sign and verify JWTs — keep this secure

8. Middleware

8.1 verifyToken

File: middlewares/verify_token.js

This is the central authentication guard applied to all protected routes. It performs two checks in sequence:

- Blacklist check — looks up the token in the BlackList MongoDB collection. If found, returns 200 with a 'You are already logout' message.
- JWT verification — calls jwt.verify() with the JWT_SECRET. On success, attaches the decoded payload to req.user and calls next(). On failure, returns 401.

The decoded req.user object (containing id and email) is used throughout the book controller to scope database queries to the authenticated user.

9. Database Configuration

File: Config/db.js

The connectDb() function establishes a Mongoose connection to MongoDB using the MONGO_URI environment variable. It is called once at application startup in index.js before the server begins listening for requests. Connection errors are logged to the console.

10. Known Limitations & Future Improvements

10.1 Current Limitations

- No input validation library (e.g., Joi or express-validator) — validation is done manually with simple checks.
- The JWT_SECRET env variable name is inconsistent: package references process.env.JWT_SECRET but the README documents SECRET_KEY.
- The route prefix /api in index.js duplicates the /api/authuser and /api/book mounts, potentially causing route conflicts.
- No pagination is implemented on GET /getBooks — all books are returned at once.
- No rate limiting on login/register endpoints, leaving them open to brute-force attempts.
- The BlackList collection is never pruned; expired tokens accumulate over time.

10.2 Suggested Improvements

- Add Joi or express-validator for robust input validation and cleaner error messages.
- Implement token expiry-aware blacklist cleanup (e.g., a scheduled job to delete expired tokens).
- Add pagination and filtering parameters to GET /getBooks.
- Implement refresh token support to allow longer sessions without re-login.
- Add rate limiting middleware (e.g., express-rate-limit) to auth endpoints.
- Write unit and integration tests using Jest and Supertest.
- Add a Swagger / OpenAPI specification for interactive API documentation.

11. Glossary

Term	Definition
JWT	JSON Web Token — a compact, URL-safe token format used for stateless authentication
bcrypt	A password-hashing algorithm that incorporates a salt to protect against rainbow table attacks
Blacklist	A MongoDB collection storing invalidated tokens to implement stateful logout
ODM	Object Document Mapper — Mongoose maps JavaScript objects to MongoDB documents
CRUD	Create, Read, Update, Delete — the four basic operations on persistent data
Bearer Token	An HTTP authentication scheme where a token string is sent in the Authorization header
Salt rounds	The cost factor for bcrypt — higher values mean slower hashing and better security
req.user	The decoded JWT payload attached to the request object by verifyToken middleware